

Introduction to Software Testing

CS 3250 Software Testing

[Ammann and Offutt, “Introduction to Software Testing,” Ch. 1, Ch. 2.1]

How Do You Test?

- Imagine we are testing a typical “add” functionality of a calculator program
- The adder() function takes 2 numbers that a user enters and returns the sum of the 2 numbers.
- The program displays the result of the adder() and then lets the user decide when the program should terminate.
- How would you **test the adder()** function? What are test cases? How many test cases do we need? How do you know?

```
def adder():  
    n1 = int(input('Enter number1: '))  
    n2 = int(input('Enter number2: '))  
    return n1 + n2  
  
end = 'n'  
while end != 'y':  
    print('sum =', adder())  
    end = input('Enter \'y\' to end the program: ')
```

More Testing & AI Tests

Refer to “[How do you test?](#)” Activity, SUT 2: Prime numbers
Design tests for the `computePrimes` method

Consider the tests you created and the tests AI generated.
What are in common?
What are different?
Is there anything missing?

(<https://www.cs.virginia.edu/~up3f/cs3250/inclass/activity-how-do-you-test.html>)

How Do You Debug?

- Something is not right about the adder() function below.
- The “add” functionality takes 2 numbers that the user enters and then displays the sum of the 2 numbers. It also allows the user to decide whether the program terminates.
- When the user enters 5 and 5, the program displays 10 as expected.
- However, when the user enters 5 and -3, the program displays 10 instead of 2.
- How would you **debug the adder()** function? What/how to fix?

```
def adder():  
    n1 = int(input('Enter number1: '))  
    n2 = int(input('Enter number2: '))  
    return n1 + n1  
  
end = 'n'  
while end != 'y':  
    print('sum =', adder())  
    end = input('Enter \'y\' to end the program: ')
```

Let's Debug a (bit) Bigger Program

Refer to “Rubber Duck Debugging” Activity on the schedule page

There is a problem with `patternIndex.java`. It fails to produce the expected output.

Rubber duck debugging



Talk to me



(<http://www.cs.virginia.edu/~up3f/cs3250/inclass/activity-debugging.html>)

Think about your experience.

What exactly is testing?

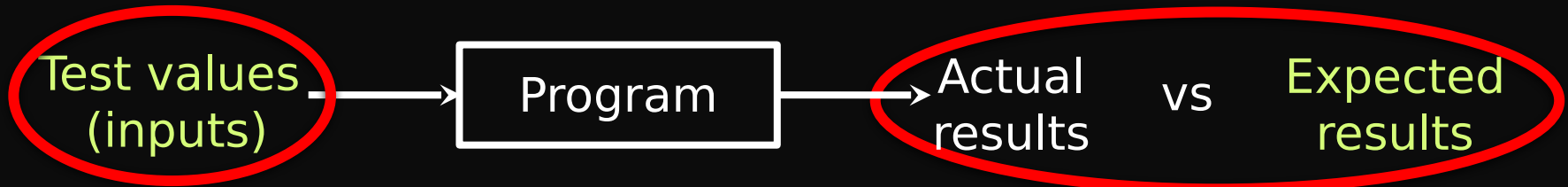
**Is testing the same as debugging?
Is debugging a testing activity?**

What is Software Testing?

- **Testing** = process of **finding input** values to check against a software (*focus of this course*)

Test case consists of **test values** and **expected results**

Goal – **reveal faults**



1. Testing is fundamentally about **choosing finite sets of values from the input domain of the software being tested**
2. Given the test inputs, compare the actual results with the expected results

- **Debugging** = process of **finding a fault** given a failure

Testing Categories

Static testing

- Testing without executing the program
 - Software inspection and some forms of analysis
 - Effective at finding certain kinds of problems such as problems that can lead to faults when the program is modified
- Inspection, walkthrough, code review, informal review, ...

Dynamic testing

- Testing by executing the program with real inputs
- Unit testing, integration testing, system testing, acceptance testing, ...

Testing Categories (2)

White-box testing

Test with **complete information** of the software (architecture, how components interact, functions and operations, code, rationale, ...)

Gray-box testing

Test with **incomplete information** of the software or **limited knowledge of internal details** (may know how components interact but not have detailed knowledge about internal program functions, source code, rationale, ...)

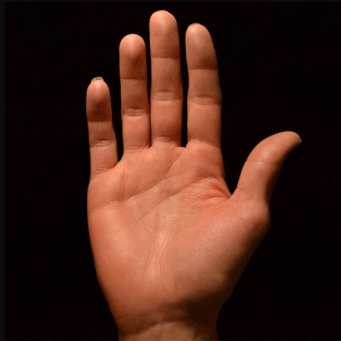
Black-box testing

Test from the outside (functionality and behavior). **Not require** source code, architecture, detailed knowledge about internal program functions, rationale, ...)

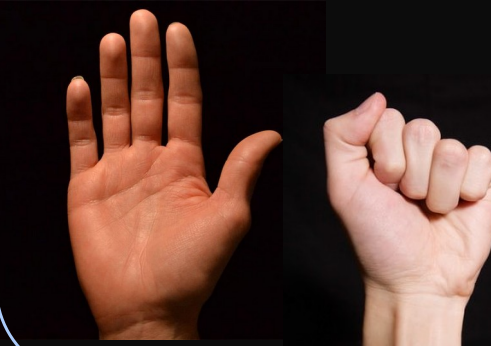
White-box vs. Black-box

- Which one should we use?

White-box?



Both?



Black-box?



[images: Left fist <https://unsplash.com/photos/ODzSyS5WsSo>
Right hand <https://www.juicyseo.co.uk/jg/wp-content/uploads/2016/02/Right-Hand.jpg>]

Testing Categories (3)

Functional testing

- Unit testing
 - Integration testing
 - System testing
 - Smoke testing
 - Acceptance testing (Alpha / Beta)
 - Agile testing
 - Regression testing
 - Continuous integration testing
- ... and many more ...

Non-functional testing

- Performance testing
 - Load testing
 - Stress testing
 - Security testing
 - Compatibility testing
 - Reliability testing
 - Usability testing
 - Compliance testing
 - Conformance testing
- ... and many more ...

Explore & share your thoughts/ideas/examples/experience/concerns on
<https://www.tldraw.com/f/xypCRh5ufr799rrrbDJsK?d=v-299.-30.1179.823.page>



(link to the tldraw also on our schedule page)

Validation and Verification (IEEE)

Validation

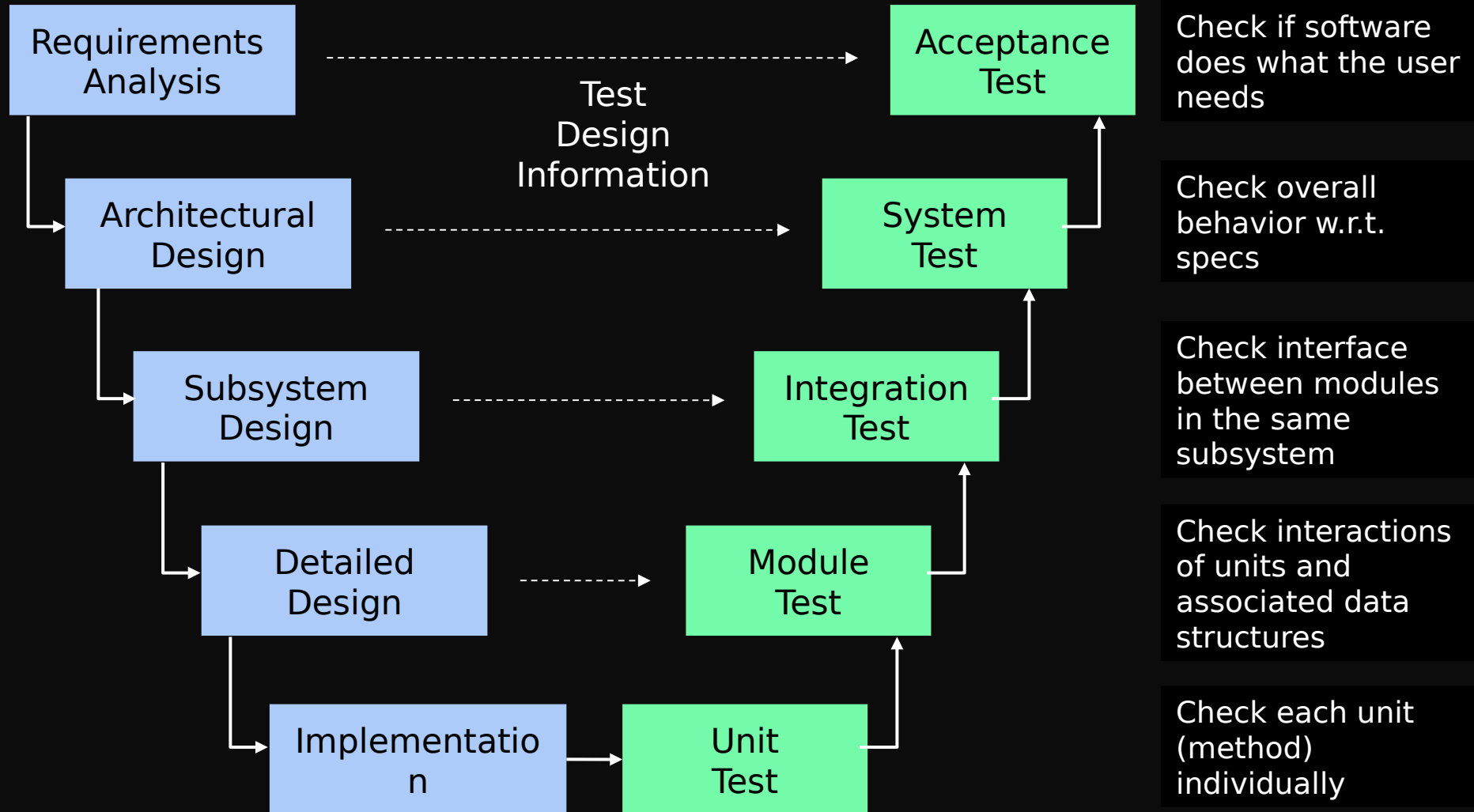
- Evaluate software **at the end** of software development
- Ensure **compliance with the intended usage**
- Done by experts in the intended usage of the software, not developers

Verification

- Evaluate software **at a given phase** of the development process
- **Fulfill the requirements** established during the previous phase
- Requires technical background on the software
- Done by developers at the various stages of development

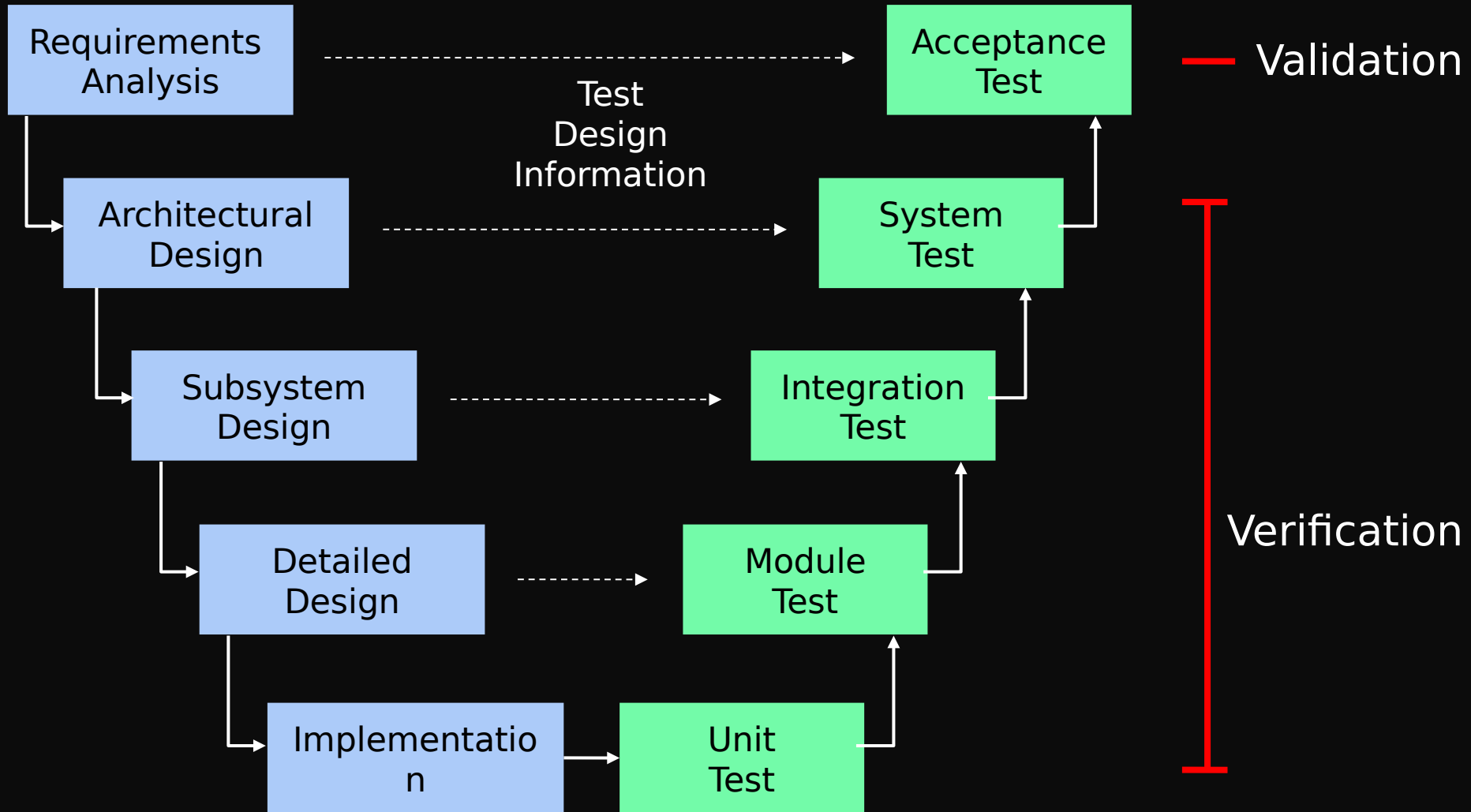
IV&V stands for “*independent verification and validation*”

Testing and SW Development Lifecycle



[based in part on AO, p.23]

Testing and SW Development Lifecycle



[based in part on AO, p.23]

Principles of Software Testing

Why testing is so hard

1. Exhaustive testing is impossible
2. We need to know when to stop testing
3. There is no silver bullet in software testing
4. Faults happen in some places more than others
5. Bug-free software does not exist
6. Testing is context-dependent
7. Verification is not validation

Tactical Goals: Each Test

*"If you don't know why you're conducting each test,
it won't be very helpful" – Jeff Offutt*

- What is objective and requirement of each test?
- What fact does each test try to verify?
- What are the threshold reliability requirements?
- What are the planned coverage levels?
- How many tests are needed?

Wrap-up

- A tester's goal is to eliminate faults as early as possible
- Testing
 - Improves software quality
 - Reduce cost
 - Preserve customer satisfaction
- **What's next?**
 - Test automation
 - JUnit